



Java Programming

(340)

STATE 2025

PRODUCTION:

AIChatBotGenerator_State

_____ (680 points)

Test Time: 90 minutes

Solution and Project (There is NO partial credit) (NOTE: UC represents uppercase and LC represents lowercase)		
The Java source file is present on the flash drive in a single folder with your contest ID		10 points
Program Execution (If the program does not execute, then the remaining items in this section receive a score of zero)		
Program loads and menu is present.		10 points
Program menu format matches the sample provided		10 points
STEP 1 Invalid Data Error is handled appropriately		20 points
STEP 1 Range Violation is handled appropriately		10 points
Option [1] is selected, program moves to appropriate STEP 2		10 points
STEP 2 Option [1] Invalid Data Error is handled appropriately		20 points
STEP 2 Option [1] Range Violation is handled appropriately		10 points
STEP 3 Option [1] correct number of random chatbots generated with all fields.		30 points
STEP 3 Option [1] program finds/prints the chatbot with “Most Expensive Complexity		30 points
STEP 3 Option [1] program finds/prints the chatbot with “Least Expensive Complexity		30 points
STEP 3 Option [1] format of output matches sample provided		20 points
After STEP 3 Option [1] Program returns immediately to active STEP 1		10 points
Option [2] is selected, program moves to appropriate STEP 2		10 points
STEP 2 Option [2] User can enter in Primary and Secondary Languages		10 points
STEP 3 Option [2] Token Rate Invalid Data Error is handled appropriately		20 points
STEP 3 Option [2] Token Rate Range Violation is handled appropriately		10 points
STEP 3 Option [2] Complexity Invalid Data Error is handled appropriately		20 points
STEP 3 Option [2] Complexity Range Violation is handled appropriately		10 points
STEP 4 Option [2] once data is properly entered program displays all bots and custom bot appears at the end of list.		30 points
After STEP 4 Option [2] Program returns immediately to active STEP 1		10 points
Option [3] is selected, program moves to appropriate STEP 2		10 points
STEP 2 Option [3] user can search for languages that have any sub string instance of their entry and returns all languages that meet the search.		30 points
After STEP 2 Option [3] Program returns immediately to active STEP 1		10 points
STEP 1 Option [4] program exits with goodbye message.		10 points
		20 points
Subtotal		/420 Points

Source Code Review No credit will be given for missing flagged comments. Code segments must work to receive full credit.		
A comment containing the contestant number is present at the top of the AIChatBotGenerator_State.java file		10 points
SC1: Flag Comment Present		5 points
SC1: Chatbots.java class: All constructors, methods, and variables are appropriately created		35 points
SC2: Flag Comment Present		5 points
SC2: AIChatBotGenerator_State class: create Scanner and ArrayList objects (place SC2 by both instances)		10 points
SC3: Flag Comment Present		5 points
SC3: AIChatBotGenerator_State class printHighestComplexity() : code that compares GPA's for all chatbots		15 points
SC4: Flag Comment Present		5 points
SC4: AIChatBotGenerator_State class printLowestComplexity() : code that compares GPA's for all chatbots		15 points
SC5: Flag Comment Present		5 points
SC5: AIChatBotGenerator_State class inputManager() : code with try/catch exception for invalid data error and range violation of STEP 1 (place SC5 by both instances)		15 points
SC6: Flag Comment Present		5 points
SC6: AIChatBotGenerator_State class inputManager() : code with try/catch exception for invalid data error and range violation of STEP 2 Option[1] (place SC6 by both instances)		15 points
SC7: Flag Comment Present		5 points
SC7: AIChatBotGenerator_State class inputManager() : code with try/catch exception for invalid INTEGER data error and range violation of STEP 3 Option[2] (place SC7 by both instances)		15 points
SC8: Flag Comment Present		5 points
SC8: AIChatBotGenerator_State class inputManager() : code with try/catch exception for invalid DOUBLE data error and range violation of STEP 3 Option[2] (place SC8 by both instances)		15 points
SC9: Flag Comment Present		5 points
SC9: AIChatBotGenerator_State class inputManager() : code that formats complexity with proper decimal format object (place SC9 by all uses of this object)		10 points
SC10: Flag Comment Present		5 points
SC10: AIChatBotGenerator_State class inputManager() : code that formats primary and secondary languages for Option [2] to required convention (first letter is UC, all other letters are LC)		15 points
SC11: Flag Comment Present		5 points
SC11: AIChatBotGenerator_State class printChatBots() : code uses printf() with all the required setting for spacing and values (see Output section for values). You must have the correct values for the field headers and the field values for the records.		35 points
Subtotal		/260 Points
Total Points		/680 Points

```

1
2 import java.io.FileNotFoundException;
3 import java.io.File;
4 import java.util.*;
5 import java.text.DecimalFormat;
6
7 public class AIChatBotGenerator_State
8 {
9
10     static Scanner sc = new Scanner(System.in); //SC2
11     static ArrayList<ChatBots> chat_Bots = new ArrayList<ChatBots>(); //SC2
12     public static void main (String args [])
13     {
14
15         inputManager(); //Given
16
17         printChatBots();
18         printHighestComplexity(); //Given
19         printLowestComplexity(); //Given
20         searchByLanguage();
21     }
22
23
24
25
26 //ChatBots will create. Signature only
27 private static void printHighestComplexity() //ArrayList<ChatBots> stuList
28 {
29     int index = 0;
30     double number = 0;
31
32     for(int i = 1; i < chat_Bots.size(); i++)
33     {
34         if(chat_Bots.get(i).getComplexity()>chat_Bots.get(index).getComplexity()) //SC3
35         {
36             index = i;
37             number = chat_Bots.get(i).getComplexity();
38         }
39     }
40
41     System.out.println("\n\nMost Expensive Complexity Rate: "+ chat_Bots.get(index).getComplexity()+ " "+chat_B
42     ots.get(index).getSupportedLanguages());
43 }
44
45 //STATE POINT
46 //ChatBots will create. Signature only
47 private static void printLowestComplexity()
48 {
49     int index = 0;
50     double number = 0;
51
52     for(int i = 1; i < chat_Bots.size(); i++)
53     {
54         if(chat_Bots.get(i).getComplexity()<chat_Bots.get(index).getComplexity()) //SC4
55         {
56             index = i;
57             number = chat_Bots.get(i).getComplexity();
58         }
59     }
60
61     System.out.println("\n\nLeast Expensive Complexity Rate: "+ chat_Bots.get(index).getComplexity()+ " "+chat_
62     Bots.get(index).getSupportedLanguages());
63 }
64
65 //Given
66 private static void setChatBots(int bC)

```

```

67 {
68     Random random = new Random();
69     ChatBots cB;
70     DecimalFormat df = new DecimalFormat("#.0000");
71     int botCount = bC;
72     double complexity;
73     int token_Rate;
74     String native_Language; String secondary_Language;
75
76     String [] supportedLanguages ={"English", "French", "Chinese", "Spanish", "Arabic", "Russian", "German", "J
apanese", "Portuguese", "Hindi", "Italian", "Dutch", "Korean", "Turkish", "Vietnamese", "Swedish", "Polish", "Indones
ian", "Greek", "Hebrew", "Thai", "Czech", "Romanian", "Hungarian", "Finnish"};
77
78     for(int i =0; i<botCount; i++)
79     {
80
81         do{
82
83             native_Language = supportedLanguages[random.nextInt(supportedLanguages.length)];
84             secondary_Language = supportedLanguages[random.nextInt(supportedLanguages.length)];
85         }while(native_Language.equals(secondary_Language));
86         token_Rate = random.nextInt(16)+5;
87         double tempComplexity = random.nextDouble()*7.1 + 2.0;
88         if(tempComplexity > 9.0) tempComplexity = Math.floor(tempComplexity);
89         String tempString =df.format(tempComplexity);
90         complexity = Double.parseDouble(tempString);
91         cB = new ChatBots(native_Language,secondary_Language,token_Rate,complexity);
92         chat_Bots.add(cB);
93     }
94 }
95
96 //Given
97 /*
98 private static void printChatBots(){
99     int i = 1;
100     for (ChatBots c : chat_Bots)
101     {
102         System.out.println(i+" "+c.getSupportedLanguages() + " | Token Rate: "+c.getTokenRate()+" | Complexity:
"+c.getComplexity());
103         i++;
104     }
105
106     printHighestComplexity();
107     printLowestComplexity();
108 }
109 */
110 private static void printChatBots() { //SC11
111     System.out.println("-----");
112     System.out.printf("| %-4s | %-30s | %-10s | %-10s |\n", "No.", "Supported Languages", "Token Rate", "Complexi
ty"); //SC11
113     System.out.println("-----");
114     int i = 1;
115     for (ChatBots c : chat_Bots) {
116         System.out.printf("| %-4d | %-30s | %-10d | %-10.4f |\n", i, c.getSupportedLanguages(), c.getTokenRate(),
c.getComplexity()); //SC11
117         i++;
118     }
119     System.out.println("-----");
120
121     printHighestComplexity();
122     printLowestComplexity();
123 }
124
125 private static void inputManager()
126 {
127     int temp =0;
128
129

```

```

130     int token_Rate =0;
131     double complexity =0.0;
132     String native_Language;
133     String secondary_Language;
134     String tempString;
135     ChatBots cB;
136     DecimalFormat df = new DecimalFormat("#.0000"); //SC9
137
138
139     while(true){
140         try{ //SC5
141             do{
142                 temp =0;
143                 System.out.println("_____");
144                 System.out.println("Welcome to the BPA Chatbot Generator. Choose one of the options:");
145                 System.out.println("Press [1] to create random chatbots OR Press [2] to create a single chatbot \n
OR Press [3] to search by language OR Press [4] to end the program");
146                 temp = sc.nextInt();
147                 if(temp <= 0 || temp > 4) System.out.println("Your entry is out of range."); //SC5
148                 sc.nextLine();
149             } while(temp <= 0 || temp > 4);
150             if(temp ==1){
151                 do{
152                     temp =0;
153                     try{ //SC6
154                         do{
155                             System.out.print("Please enter in a value between 1 and 50: ");
156                             temp = sc.nextInt();
157                             if(temp >50 || temp <1) System.out.println("Your entry is out of range."); //SC6
158                             sc.nextLine();
159                         }while(temp>50|| temp<1);
160                     }
161                     catch(InputMismatchException e) //SC6
162                     {
163                         sc.next();
164                         System.out.println("\nERROR: Please enter valid data type.");
165                     }
166                 } while(temp >50 || temp <1);
167                 setChatBots(temp);
168                 temp =0;
169                 System.out.println("_____");
170                 printChatBots();
171             }
172             if(temp ==2){
173
174                 System.out.println("You are now creating a single chatbot");
175                 System.out.println("Enter in the native language");
176                 native_Language = sc.nextLine().toLowerCase();
177
178                 System.out.println("Enter in the secondary language");
179                 secondary_Language = sc.nextLine().toLowerCase();
180                 do{
181                     try{ //SC7
182                         do{
183                             System.out.println("Enter the token rate value (5 to 20): ");
184                             token_Rate = sc.nextInt();
185                             if(token_Rate >20 || token_Rate <5) System.out.println("Your entry is out of range."); //
SC7
186                             }while(token_Rate >20 || token_Rate <5);
187                         }
188                         catch(InputMismatchException e) //SC7
189                         {
190                             sc.next();
191                             System.out.println("\nERROR: Please enter valid data type.");
192                         }
193                     }while(token_Rate >20 || token_Rate <5);
194                     do{
195                         try{ //SC8

```

```

196         do{
197             System.out.println("Enter in the token complexity in its proper format (###.###) in a range
of 2.0 to 9.0");
198             complexity = sc.nextDouble();
199             tempString = df.format(complexity); //SC9
200             complexity = Double.parseDouble(tempString);
201             if(complexity>9.0 || complexity < 2.0) System.out.println("Your entry is out of range.");
//SC8
202         }while(complexity>9.0 || complexity < 2.0);
203     }
204     catch(InputMismatchException e) //SC8
205     {
206         sc.next();
207         System.out.println("\nERROR: Please enter valid data type.");
208     }
209     }while(complexity>4.0 || complexity <= 0.0);
210     native_Language = native_Language.substring(0,1).toUpperCase() + native_Language.substring(1).toLo
werCase(); //SC10
211     secondary_Language = secondary_Language.substring(0,1).toUpperCase() + secondary_Language.substrin
g(1).toLowerCase(); //SC10
212     cB = new ChatBots(native_Language,secondary_Language,token_Rate,complexity);
213     chat_Bots.add(cB);
214     System.out.println("_____");
215     printChatBots();
216 }
217 if(temp ==3 ){
218     if (chat_Bots.size()==0){
219         System.out.println("You must create chatbots first.");
220     } else {searchByLanguage();}
221 }
222 }
223
224 if(temp == 4){
225     System.out.println("Goodbye!");
226     System.exit(0);
227 }
228 }
229 catch(InputMismatchException e) //SC5
230 {
231     sc.next();
232     System.out.println("\nERROR: Please enter valid data type.");
233 }
234 }
235 }
236 }
237 private static void searchByLanguage() {
238     System.out.println("Enter the language you want to search for:");
239     String searchLanguage = sc.nextLine().toLowerCase();
240
241     boolean found = false;
242     for (ChatBots bot : chat_Bots) {
243         if (bot.getSupportedLanguages().toLowerCase().contains(searchLanguage)) {
244             System.out.println("Chatbot found: " + bot.getSupportedLanguages() + " | Token Rate: " + bot.getToken
Rate() + " | Complexity: " + bot.getComplexity());
245             found = true;
246         }
247     }
248
249     if (!found) {
250         System.out.println("No chatbot found for the specified language.");
251     }
252 }
253 }
254 }
255
256 //SC1
257 class ChatBots
258 {

```

```

259 //Create appropriately variables based upon constructor parameters
260 String native_Language;
261 String secondary_Language;
262 int token_Rate;
263 double complexity;
264 //Generic Constructor
265 public ChatBots()
266 {
267     native_Language = "Esparanto";
268     secondary_Language = "Java";
269     token_Rate = 0;
270     complexity = 0.0;
271 }
272 //Constructor with Parameters
273 public ChatBots(String nL, String sL, int tR, double co)
274 {
275     native_Language = nL;
276     secondary_Language = sL;
277     token_Rate = tR;
278     complexity = co;
279 }
280
281 //Returns "last name, first name" with appropriate variables
282 public String getSupportedLanguages()
283 {
284     return secondary_Language + ", "+native_Language;
285 }
286 //Returns first name
287 public String getNativeLanguage()
288 {
289     return native_Language;
290 }
291 //Returns last name
292 public String getSecondaryLanguage()
293 {
294     return secondary_Language;
295 }
296 //Returns the Complexity
297 public double getComplexity()
298 {
299     return complexity;
300 }
301 //Returns token_Rate level
302 public int getTokenRate()
303 {
304     return token_Rate;
305 }
306
307
308 }

```